

First Project with Open-RMF

Creating ROS packages

Create a ROS workspace

```
mkdir -p /home/usuario/rmf_ws/src
```

Launch terminal in Open-RMF container

```
cd /home/usuario
```

```
rocker --nvidia --x11 --name rmf_demos \  
  -e ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST \  
  --network host --user \  
  --volume `pwd`/rmf_ws:/home/usuario/rmf_ws --\  
  ghcr.io/open-rmf/rmf/rmf_demos:jazzy-rmf-latest \  
  bash
```

Check the office simulation

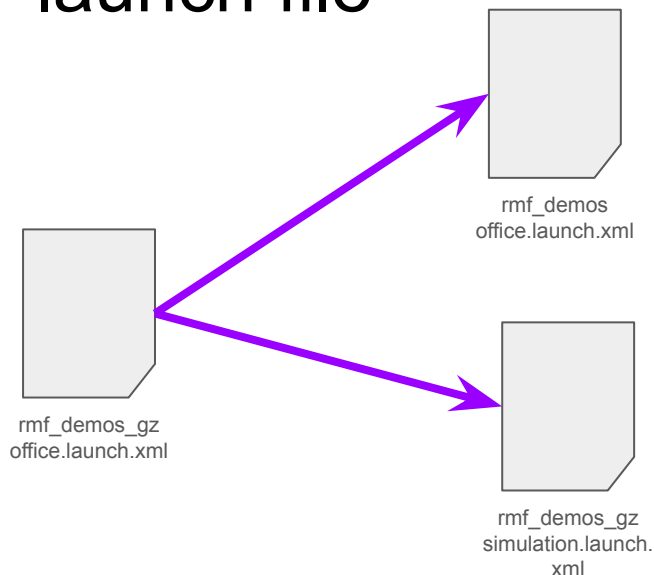
```
sudo cp -R /root/.gazebo .  
ros2 launch rmf_demos_gz \  
    office.launch.xml
```

Task: create a ROS package for the first project

Create a ROS package for the first project with all the necessary files for building the world and launching the simulation with a fleet of Tinyrobots.

To do so, let's examine the files used in the simulation of the office world of the `rmf_demos` package.

Structure of the RMF office launch file



```
<?xml version='1.0' ?>
```

```
<launch>
```

```
<arg name="use_sim_time" default="true"/>
<arg name="failover_mode" default="false"/>
<arg name="gazebo_version" default='8'/>
<arg name="sim_update_rate" default='100'/>
```

```
<!-- Common launch -->
```

```
<include file="$(find-pkg-share rmf_demos)/office.launch.xml">
  <arg name="use_sim_time" value="$(var use_sim_time)"/>
  <arg name="failover_mode" value="$(var failover_mode)"/>
</include>
```

```
<!-- Simulation launch -->
```

```
<include file="$(find-pkg-share rmf_demos_gz)/simulation.launch.xml">
  <arg name="map_name" value="office" />
  <arg name="gazebo_version" value="$(var gazebo_version)" />
  <arg name="sim_update_rate" value="$(var sim_update_rate)"/>
</include>
```

```
</launch>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos_gz/launch/office.launch.xml

```

<?xml version='1.0' ?>

<launch>
  <arg name="map_package" default="rmf_demos_maps" description="Name of the map package" />
  <arg name="map_name" description="Name of the rmf_demos map to simulate" />
  <arg name="use_crowdsim" default='0' />
  <arg name="gazebo_version" default='8' />
  <arg name="sim_update_rate" default='100' />

  <let name="world_path" value="$(find-pkg-share $(var map_package))/maps/$(var map_name)/$(var map_name).world" />
  <let name="model_path" value="$(find-pkg-share $(var map_package))/maps/$(var map_name)/models:$(env HOME)/.gazebo/models" />

  <!-- Use crowd sim if `use_crowdsim` is true -->
  <let name="menge_resource_path" if="$(var use_crowdsim)" value="$(find-pkg-share $(var map_package))/maps/$(var map_name)/config_resource"/>
  <let name="menge_resource_path" unless="$(var use_crowdsim)" value="" />

  <let name="gz_headless" if="$(var headless)" value="-s"/>
  <let name="gz_headless" unless="$(var headless)" value="" />

  <!-- TODO(luca) Remove the manual concatenation of GZ SIM_RESOURCE_PATH and just use environment hooks -->
  <executable cmd="gz sim --force-version $(var gazebo_version) $(var gz_headless) -r -v 3 $(var world_path) -z $(var sim_update_rate)" output="both">
    <env name="GZ_SIM_RESOURCE_PATH" value="$(env GZ_SIM_RESOURCE_PATH):$(var model_path)" />
    <env name="MENGE_RESOURCE_PATH" value="$(var menge_resource_path)"/>
  </executable>

  <!-- ros_gz bridge for simulation clock -->
  <node pkg="ros_gz_bridge" exec="parameter_bridge"
    args="/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock]
    />

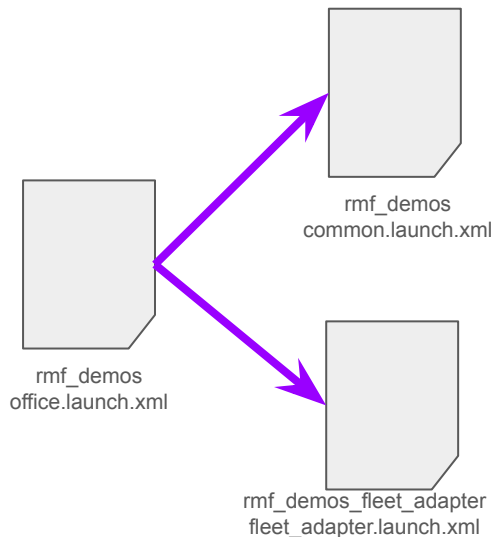
</launch>

```

Simulation launch

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos_gz/launch/simulation.launch.xml

Common launch (i)



```
<?xml version='1.0' ?>
```

```
<launch>
```

```
  <arg name="use_sim_time" default="false"/>
```

```
  <!-- Common launch -->
```

```
  <include file="$(find-pkg-share rmf_demos)/common.launch.xml">
```

```
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
```

```
    <arg name="viz_config_file" value="$(find-pkg-share rmf_demos)/include/office/office.rviz"/>
```

```
    <arg name="config_file" value="$(find-pkg-share rmf_demos_maps)/office/office.building.yaml"/>
```

```
    <arg name="use_reservation_node" value="true"/>
```

```
  </include>
```

```
  <!-- TinyRobot fleet adapter -->
```

```
  <group>
```

```
    <include file="$(find-pkg-share rmf_demos_fleet_adapter)/launch/fleet_adapter.launch.xml">
```

```
      <arg name="use_sim_time" value="$(var use_sim_time)"/>
```

```
      <arg name="nav_graph_file" value="$(find-pkg-share rmf_demos_maps)/maps/office/nav_graphs/0.yaml" />
```

```
      <arg name="config_file" value="$(find-pkg-share rmf_demos)/config/office/tinyRobot_config.yaml"/>
```

```
    </include>
```

```
  </group>
```

```
</launch>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos/launch/office.launch.xml

Common launch (ii)

```
<?xml version='1.0' ?>
```

```
<launch>
```

```
<arg name="use_sim_time" default="false" description="Use the /clock topic for time to sync with simulation"/>
<arg name="viz_config_file" default="$(find-pkg-share rmf_visualization_schedule)/config/rmf.rviz"/>
<arg name="config_file" description="Building description file required by rmf_building_map_tools"/>
<arg name="initial_map" default="L1" description="Initial map name for the visualizer"/>
<arg name="headless" default="false" description="do not launch rviz; launch gazebo in headless mode"/>
<arg name="bidding_time_window" description="Time window in seconds for task bidding process" default="2.0"/>
<arg name="use_unique_hex_string_with_task_id" default="true" description="Appends a unique hex string to the task ID"/>
<arg name="server_uri" default="" description="The URI of the api server to transmit state and task information."/>
<arg name="use_reservation_node" default="false" description="Enable the reservation node for parking spot de-confliction"/>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos/launch/common.launch.xml

Common launch (iii)

```
<!-- Traffic Schedule -->
<node pkg="rmf_traffic_ros2" exec="rmf_traffic_schedule" output="both" name="rmf_traffic_schedule_primary">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
</node>

<!-- Blockade Moderator -->
<node pkg="rmf_traffic_ros2" exec="rmf_traffic_blockade" output="both">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
</node>

<!-- Building Map -->
<group>
  <node pkg="rmf_building_map_tools" exec="building_map_server" args="$(var config_file)">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos/launch/common.launch.xml

Common launch (iv)

```
<!-- Visualizer -->
<group>
  <include file="$(find-pkg-share rmf_visualization)/visualization.launch.xml">
    <arg name="use_sim_time" value="$(var use_sim_time)"/>
    <arg name="map_name" value="$(var initial_map)"/>
    <arg name="viz_config_file" value="$(var viz_config_file)"/>
    <arg name="headless" value="$(var headless)"/>
  </include>
</group>

<!-- Door Supervisor -->
<group>
  <node pkg="rmf_fleet_adapter" exec="door_supervisor">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>

<!-- Lift Supervisor -->
<group>
  <node pkg="rmf_fleet_adapter" exec="lift_supervisor">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
  </node>
</group>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos/launch/common.launch.xml

Common launch (v)

```
<!-- Dispatcher Node -->
<group>
  <node pkg="rmf_task_ros2" exec="rmf_task_dispatcher" output="screen">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
    <param name="bidding_time_window" value="$(var bidding_time_window)"/>
    <param name="use_unique_hex_string_with_task_id" value="$(var use_unique_hex_string_with_task_id)"/>
    <param name="server_uri" value="$(var server_uri)"/>
  </node>
</group>

<!-- Proto-Reservation system-->
<group if="$(var use_reservation_node)">
  <node pkg="rmf_reservation_node" exec="queue_manager"></node>
</group>

</launch>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos/launch/common.launch.xml

Visualization (i)

```
<?xml version = '1.0' ?>
```

```
<launch>
```

```
  <arg name="use_sim_time" default="false"/>
  <arg name="rate" default="10.0" description="The rate in hz at which the scheduler markers should publish"/>
  <arg name="map_name" default="B1" description="The name of the map to initialize the visualizer"/>
  <arg name="viz_config_file" default="$(find-pkg-share rmf_visualization_schedule)/config/rmf.rviz"/>
  <arg name="display_names" default="true"/>
  <arg name="websocket_port" default="8006"/>
  <arg name="headless" default="false" description="Do not launch rviz2"/>
  <arg name="path_width" default="0.2" description="Width of schedule path(m) to be rendered"/>
  <arg name="lane_width" default="0.5" description="Width of lanes(m) to be rendered"/>
  <arg name="waypoint_scale" default="1.3" description="The relative size of a waypoint wrt the lane width"/>
  <arg name="text_scale" default="0.7" description="The relative size of a text label wrt the lane width"/>
  <arg name="lane_transparency" default="0.6" description="The transparency of a rendered lane in default state"/>
  <arg name="wait_secs" default="10" description="The number of seconds the schedule visualizer should keep trying to connect to the rmf_schedule_node"/>
  <arg name="retained_history_count" default="50" description="The retained history count for negotiations"/>
  <arg name="fleet_state_nose_scale" default="0.5" description="The radius of the nose marker as a fraction of the fleet's footprint radius"/>
  <arg name="rmf_frame_id" default="map" description="The RMF global frame id"/>
```

https://github.com/open-rmf/rmf_visualization/blob/main/rmf_visualization/launch/visualization.launch.xml

Visualization (ii)

```
<group>
  <node pkg="rmf_visualization_schedule" exec="schedule_visualizer_node">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
    <param name="rate" value="$(var rate)"/>
    <param name="path_width" value="$(var path_width)"/>
    <param name="initial_map_name" value="$(var map_name)"/>
    <param name="wait_secs" value="$(var wait_secs)"/>
    <param name="port" value="$(var websocket_port)"/>
    <param name="retained_history_count" value="$(var retained_history_count)"/>
  </node>

  <node pkg="rmf_visualization_fleet_states" exec="fleetstates_visualizer_node">
    <param name="use_sim_time" value="$(var use_sim_time)"/>
    <param name="fleet_state_nose_scale" value="$(var fleet_state_nose_scale)"/>
    <!-- Radius parameters for fleets available in rmf_demos -->
    <param name="tinyRobot_radius" value="0.3"/>
    <param name="deliveryRobot_radius" value="0.6"/>
    <param name="cleanerBotA_radius" value="1.0"/>
    <param name="cleanerBotE_radius" value="1.0"/>
    <param name="caddy_radius" value="1.5"/>
  </node>
```

https://github.com/open-rmf/rmf_visualization/blob/main/rmf_visualization/launch/visualization.launch.xml

Visualization (iii)

```
<node pkg="rmf_visualization_building_systems" exec="rmf_visualization_building_systems" args="-m $(var map_name)">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
</node>
```

```
<node pkg="rmf_visualization_navgraphs" exec="navgraph_visualizer_node">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="initial_map_name" value="$(var map_name)"/>
  <param name="lane_width" value="$(var lane_width)"/>
  <param name="waypoint_scale" value="$(var waypoint_scale)"/>
  <param name="text_scale" value="$(var text_scale)"/>
  <param name="lane_transparency" value="$(var lane_transparency)"/>
</node>
```

https://github.com/open-rmf/rmf_visualization/blob/main/rmf_visualization/launch/visualization.launch.xml

Visualization (iv)

```
<node pkg="rmf_visualization_obstacles" exec="obstacle_visualizer_node">
  <param name="use_sim_time" value="$(var use_sim_time)"/>
  <param name="initial_map_name" value="$(var map_name)"/>
  <param name="global_fixed_frame" value="$(var rmf_frame_id)"/>
</node>
</group>

<group unless="$(var headless)">
  <executable cmd="rviz2 -d $(var viz_config_file)" output="both"/>
</group>
```


Fleet adapter launch (i)

```
<?xml version='1.0' ?>

<launch>

  <arg name="use_sim_time" default="true" description="Use the /clock topic for time to sync with simulation"/>
  <arg name="config_file" description="The config file that provides important parameters for setting up the adapter"/>
  <arg name="nav_graph_file" description="The graph that this fleet should use for navigation"/>
  <arg name="server_uri" default="" description="The URI of the api server to transmit state and task information."/>
  <arg name="output" default="screen"/>

  <!-- Fleet manager -->
  <node pkg="rmf_demos_fleet_adapter"
        exec="fleet_manager"
        args="--config_file $(var config_file)"
        output="both">

    <param name="use_sim_time" value="$(var use_sim_time)"/>

  </node>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos_fleet_adapter/launch/fleet_adapter.launch.xml

Fleet adapter launch (ii)

```
<!-- Fleet adapter -->
<group>
  <group if="$(var use_sim_time)">
    <node pkg="rmf_demos_fleet_adapter"
      exec="fleet_adapter"
      args="-c $(var config_file) -n $(var nav_graph_file) -sim"
      output="both">

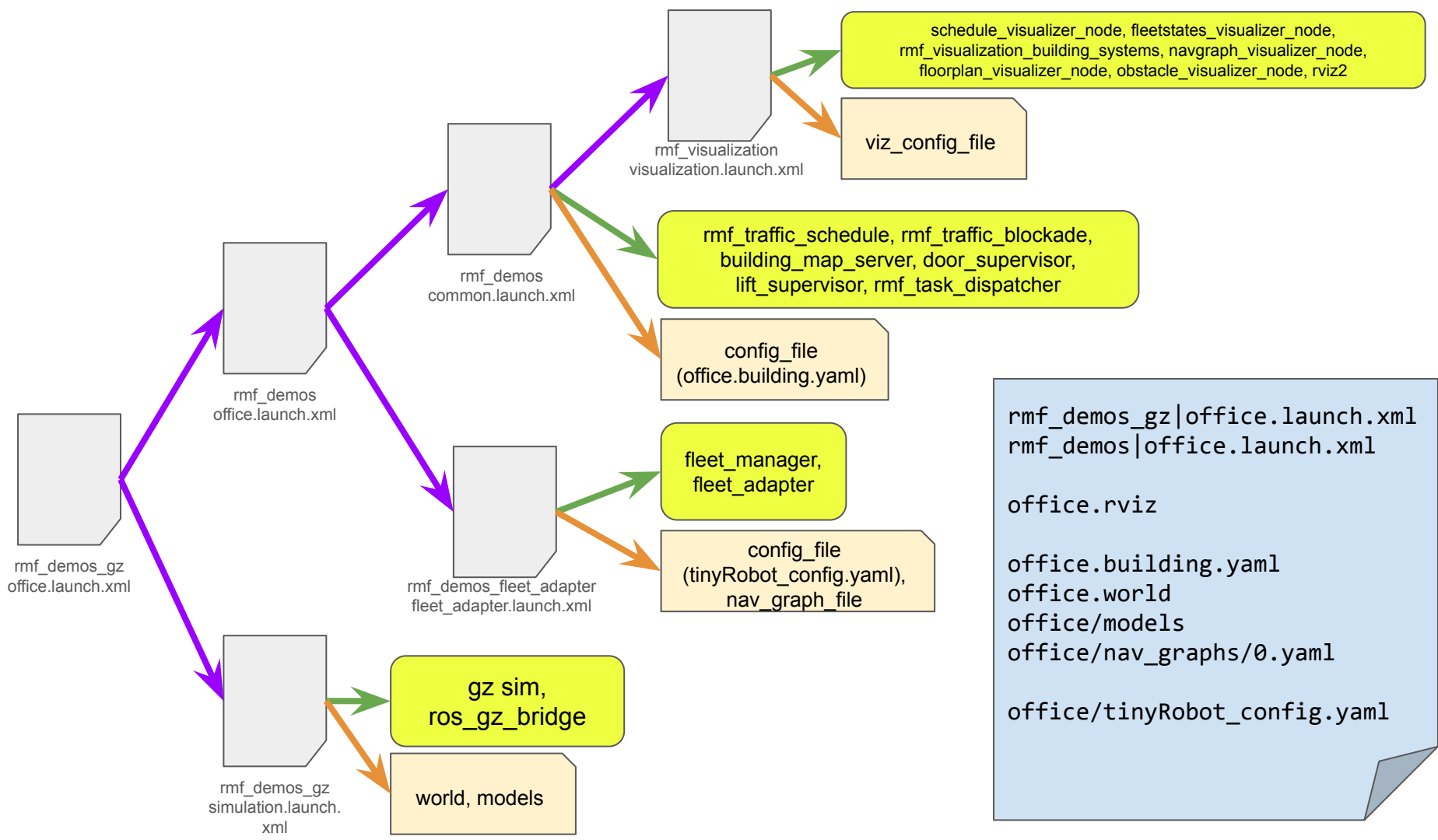
    <param name="use_sim_time" value="$(var use_sim_time)"/>
    <param name="server_uri" value="$(var server_uri)"/>
  </node>
</group>

<group unless="$(var use_sim_time)">
  <node pkg="rmf_demos_fleet_adapter"
    exec="fleet_adapter"
    args="-c $(var config_file) -n $(var nav_graph_file)"
    output="both">

    <param name="use_sim_time" value="$(var use_sim_time)"/>
    <param name="server_uri" value="$(var server_uri)"/>
  </node>
</group>
</group>

</launch>
```

https://github.com/open-rmf/rmf_demos/blob/main/rmf_demos_fleet_adapter/launch/fleet_adapter.launch.xml



rmf_demos_gz | office.launch.xml
rmf_demos | office.launch.xml

office.rviz

office.building.yaml
office.world
office/models
office/nav_graphs/0.yaml
office/tinyRobot_config.yaml

Packages and files

rmf_demos_gz	launch/office.launch.xml
rmf_demos	launch/office.launch.xml
rmf_demos	launch/include/office/office.rviz
rmf_demos_maps	maps/office/office.building.yaml
install	share/rmf_demos_maps/maps/office/office.world
	share/rmf_demos_maps/maps/office/models
	share/rmf_demos_maps/maps/office/nav_graphs/0.yaml
rmf_demos	config/office/tinyRobot_config.yaml

Exercise

Create a ROS package with a new Open-RMF world, including a robot.

First create a simple world according to the following slides.

Then create a second world with the ICC Kyoto model including robots.

Refer to the [RMF demos packages](#) for building the models, and creating all the necessary launch files.

[RUA: Practical guide to implement OpenRMF in multi-robot environments](#)

Practical guide to implement OpenRMF in multi-robot environments

Grado en Ingeniería Informática



Trabajo Fin de Grado

Autor:

Joel Romero Mollá

Tutor/es:

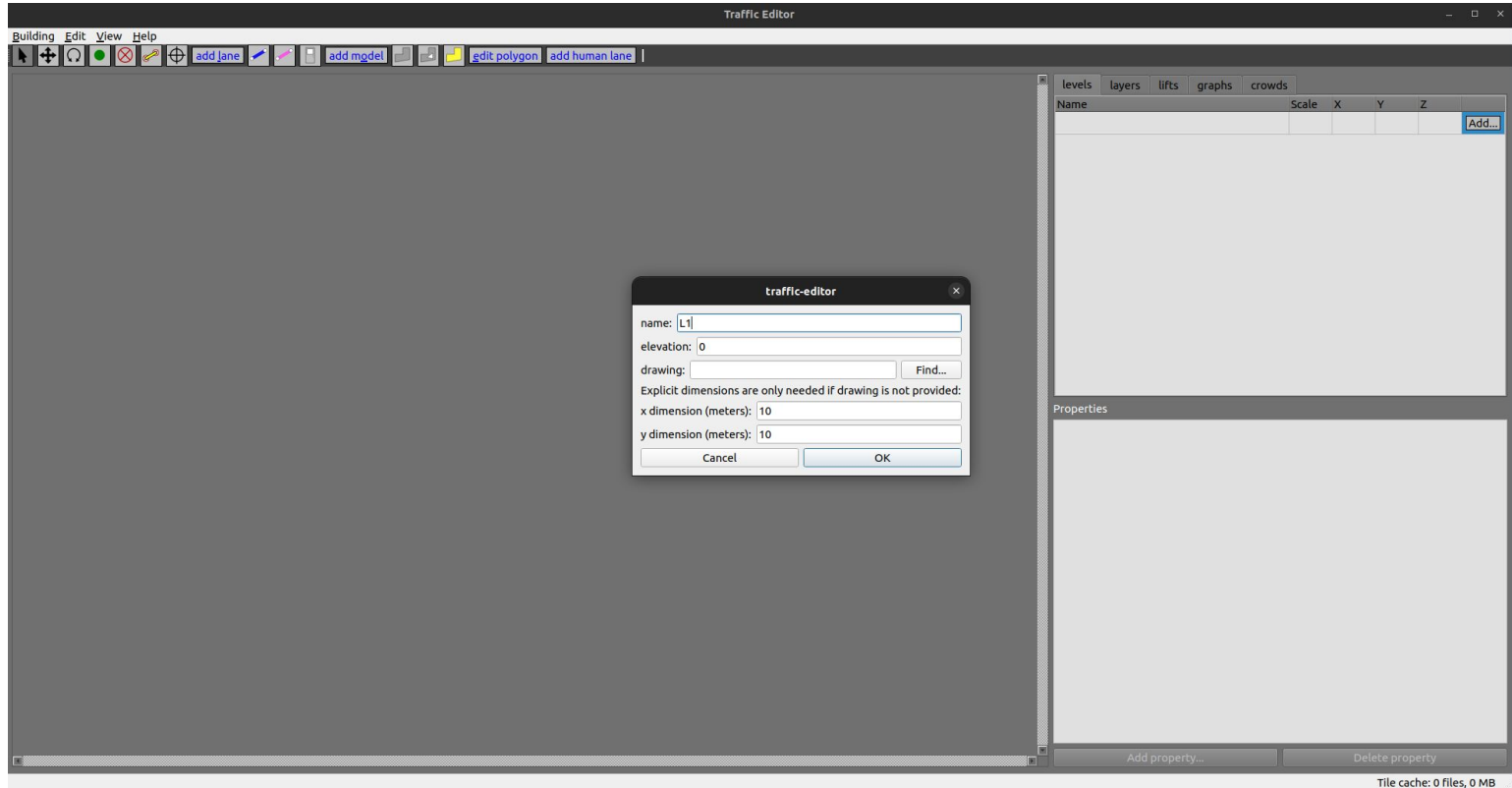
María del Pilar Arques Corrales



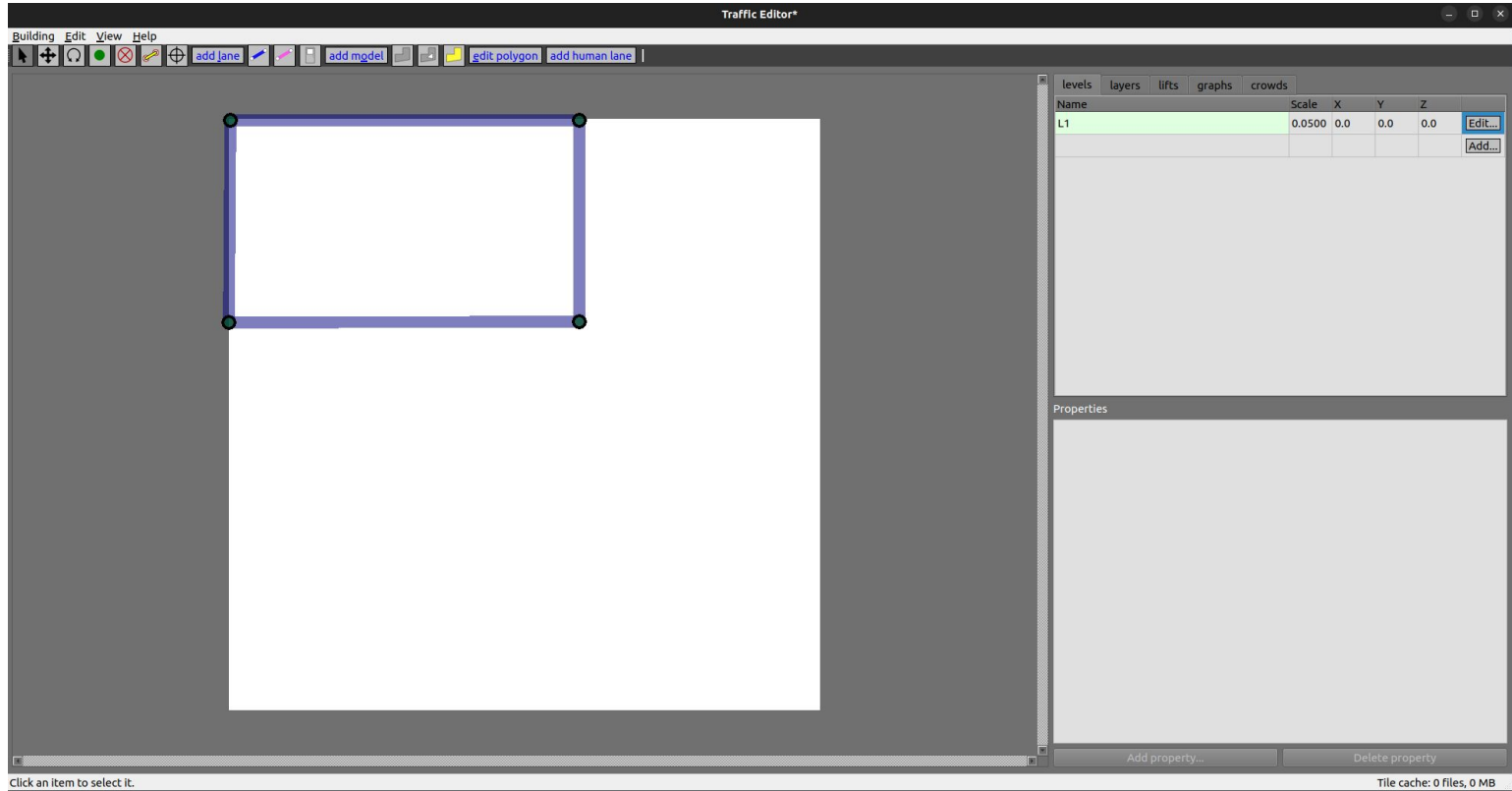
Universitat d'Alacant
Universidad de Alicante

Julio 2024

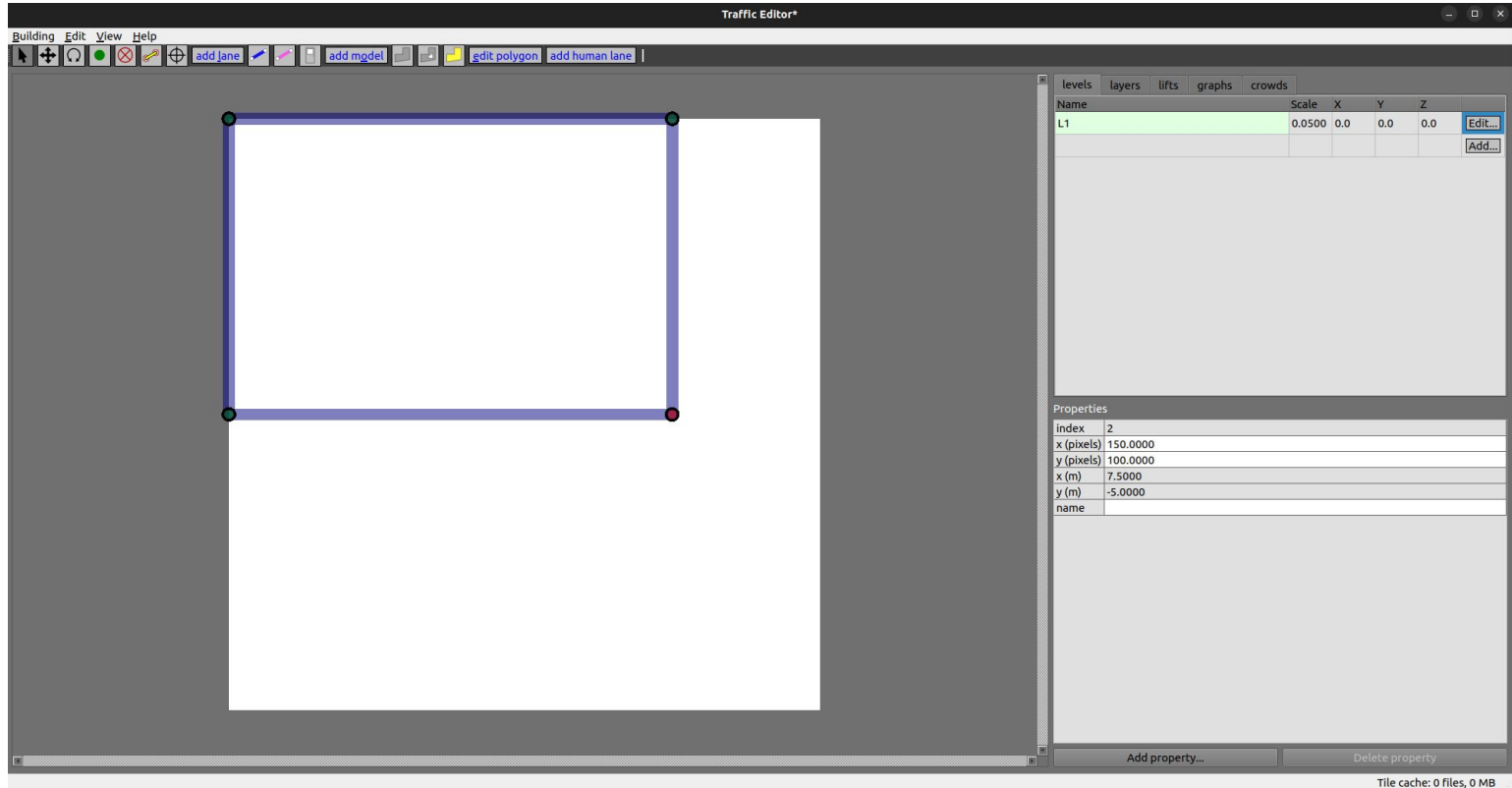
Create a map



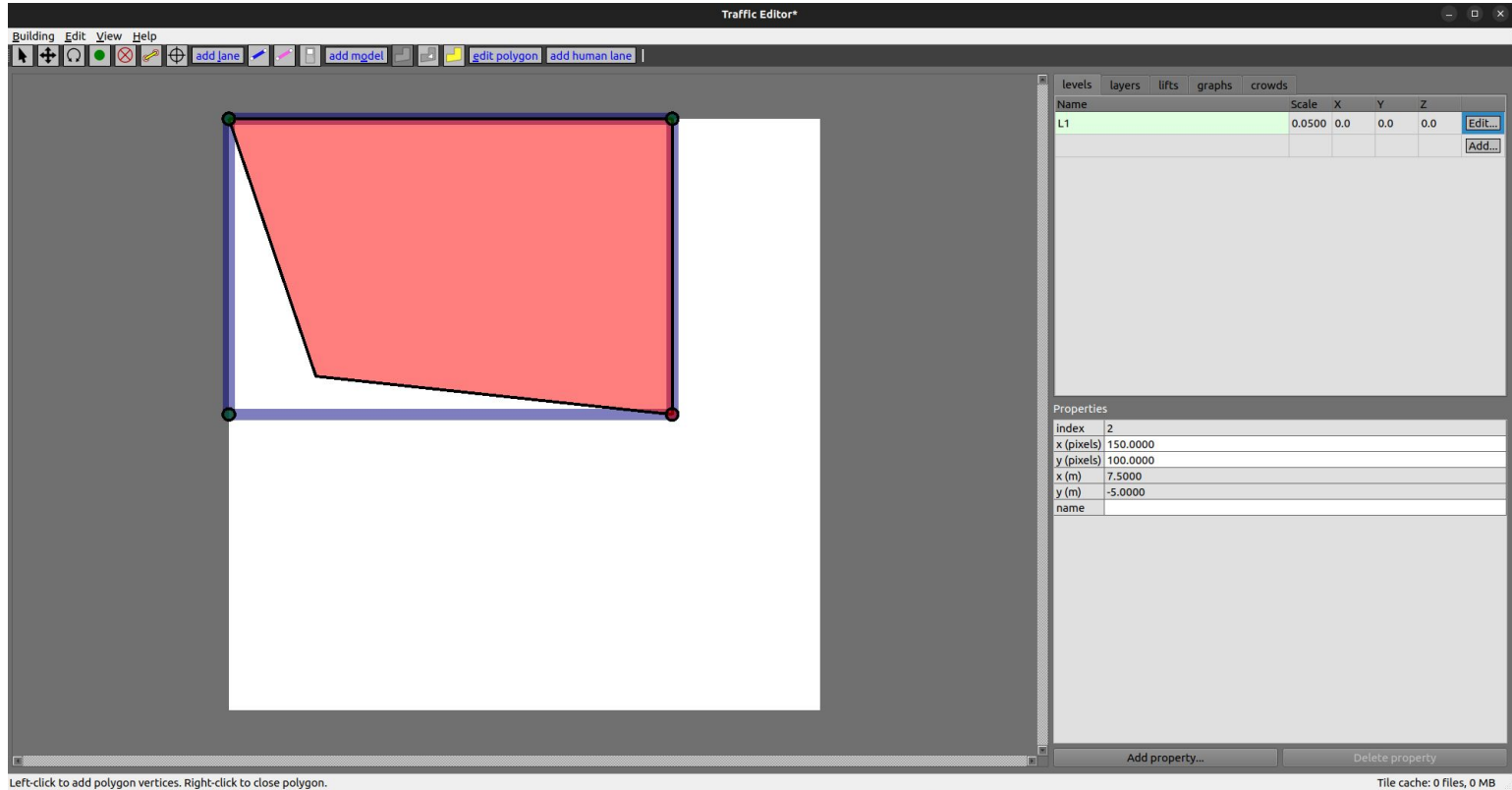
Add walls



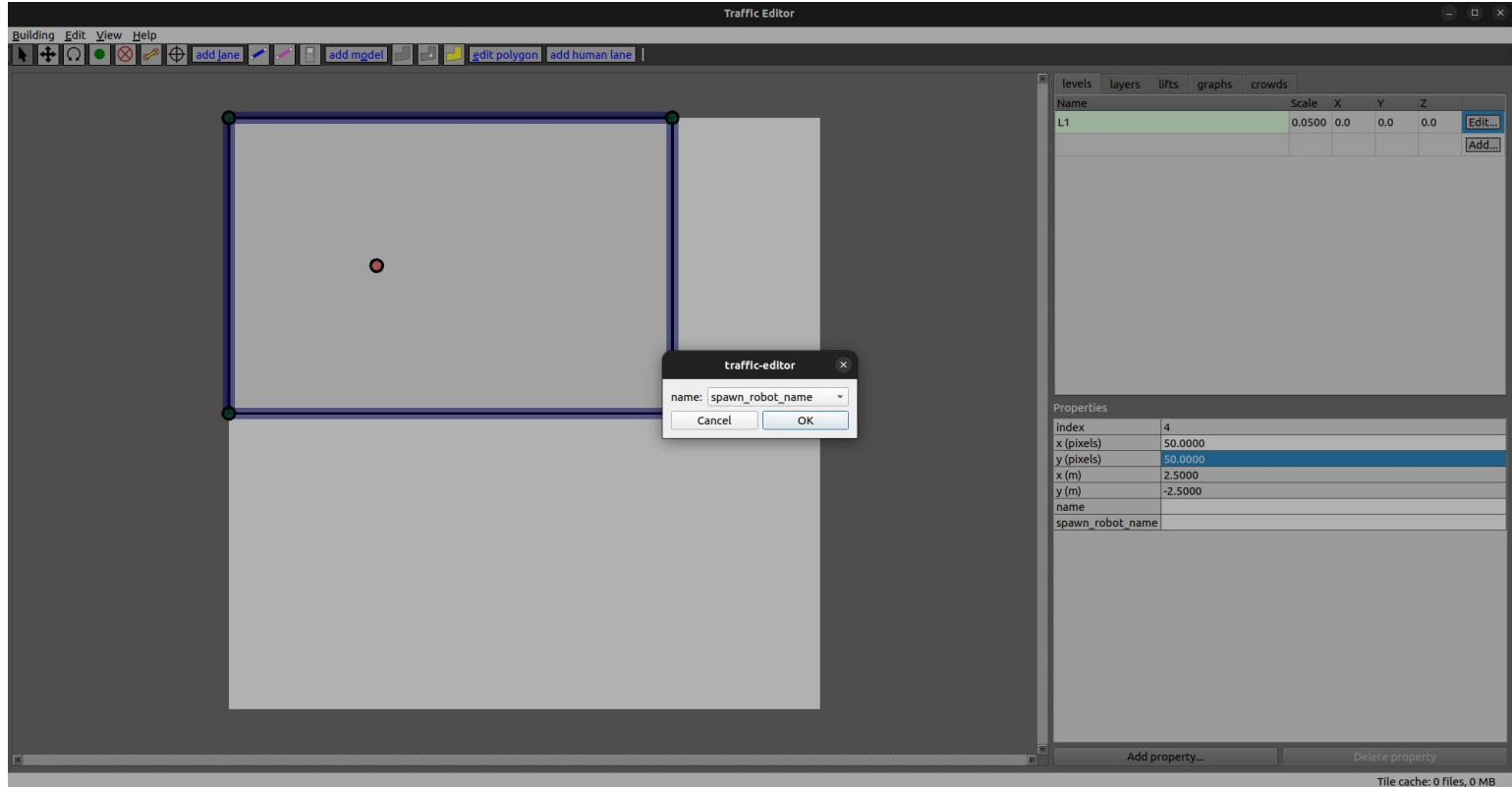
Align walls



Add a floor



Spawn a robot

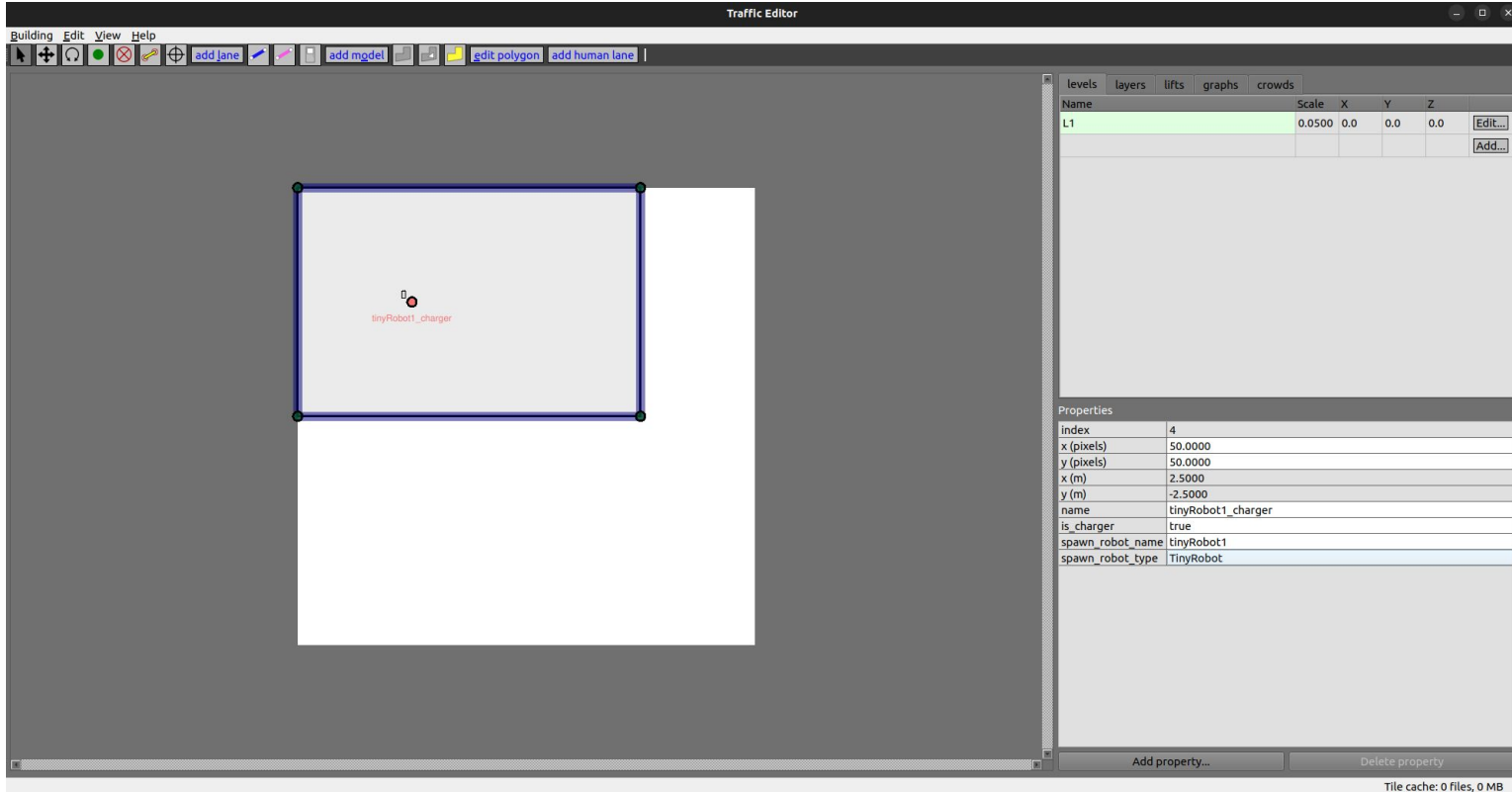


Robot configuration

Traffic Editor

Building Edit View Help

add lane add mgdel edit polygon add human lane



Name	Scale	X	Y	Z
L1	0.0500	0.0	0.0	0.0

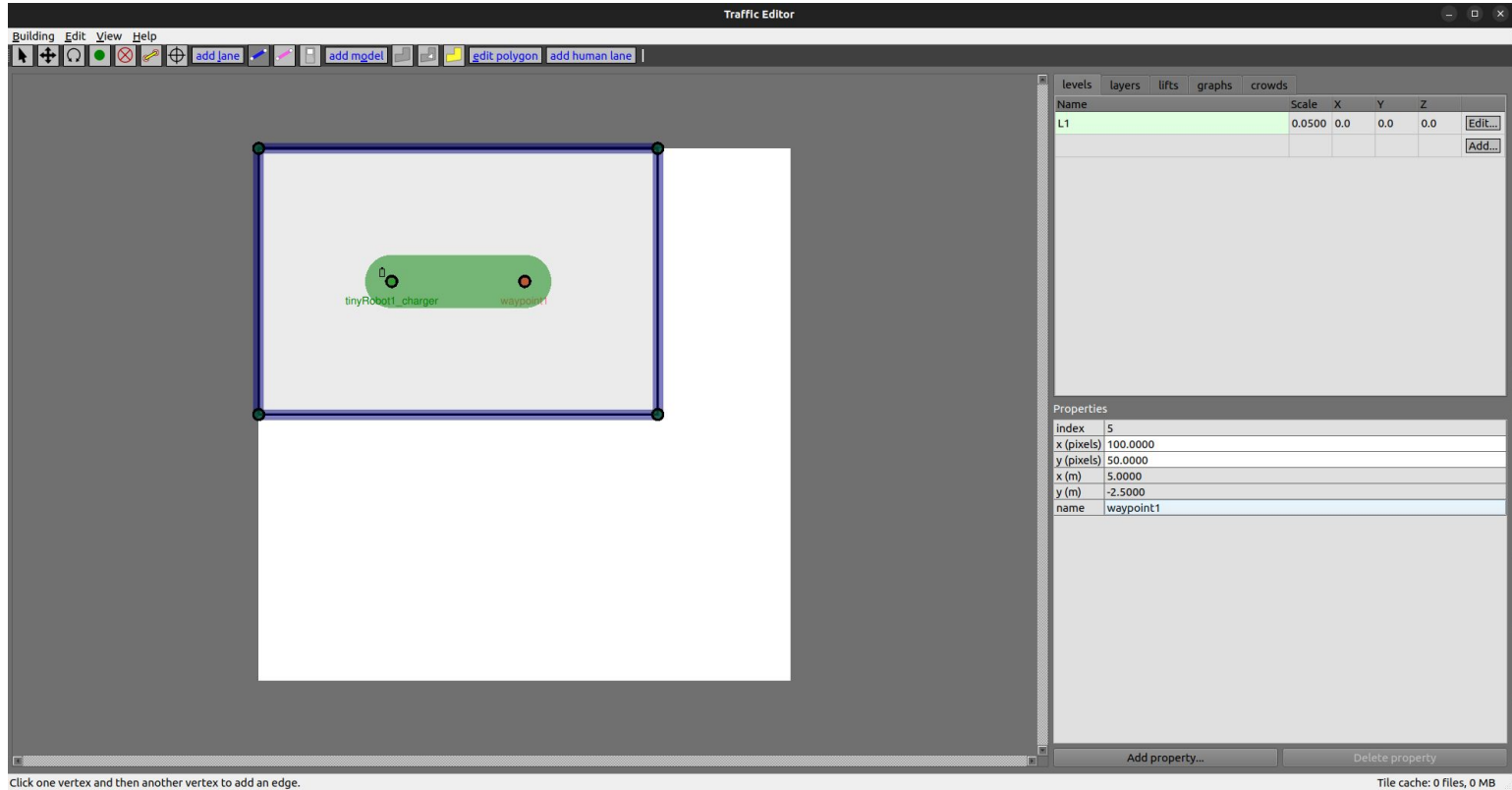
Properties

index	4
x (pixels)	50.0000
y (pixels)	50.0000
x (m)	2.5000
y (m)	-2.5000
name	tinyRobot1_charger
is_charger	true
spawn_robot_name	tinyRobot1
spawn_robot_type	TinyRobot

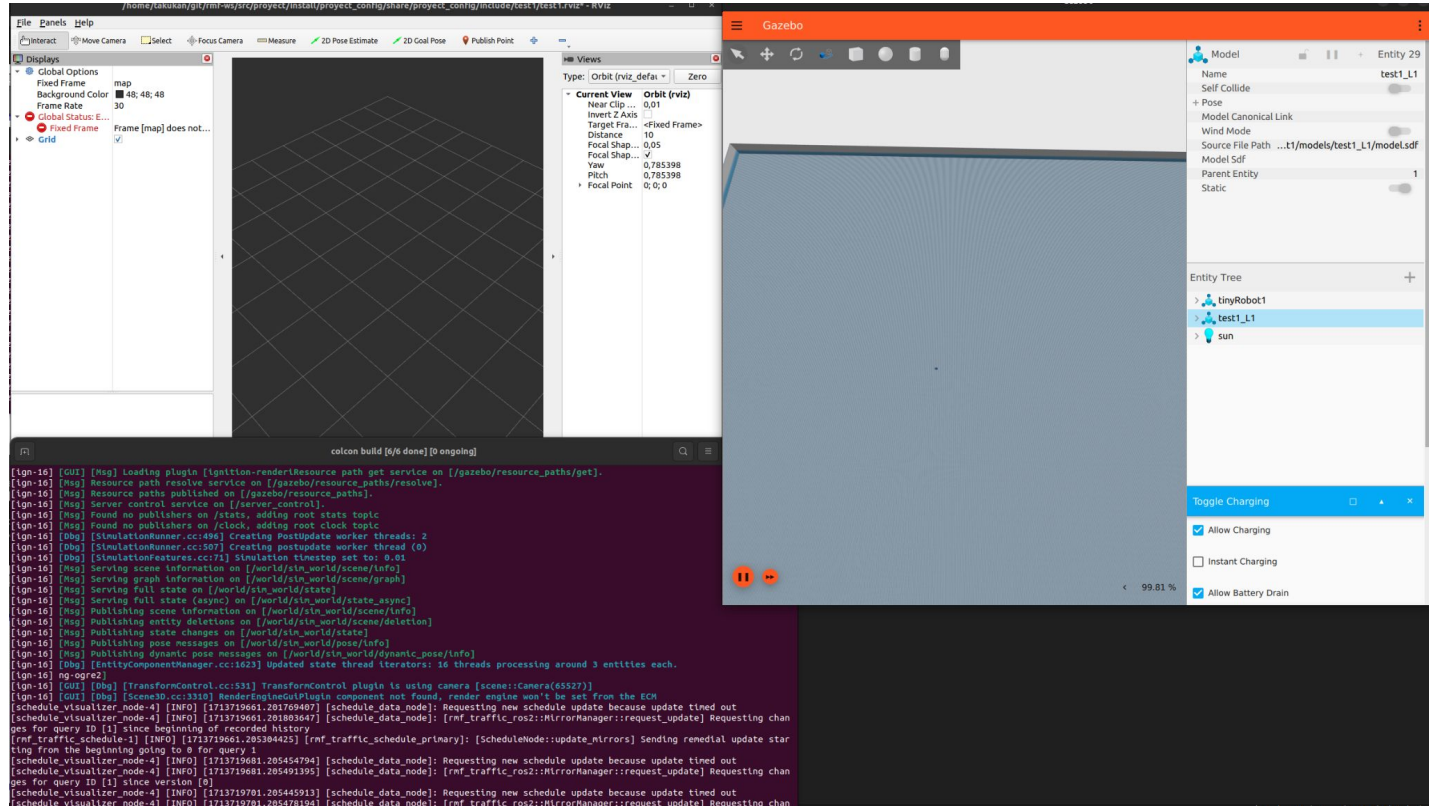
Add property... Delete property

Tile cache: 0 files, 0 MB

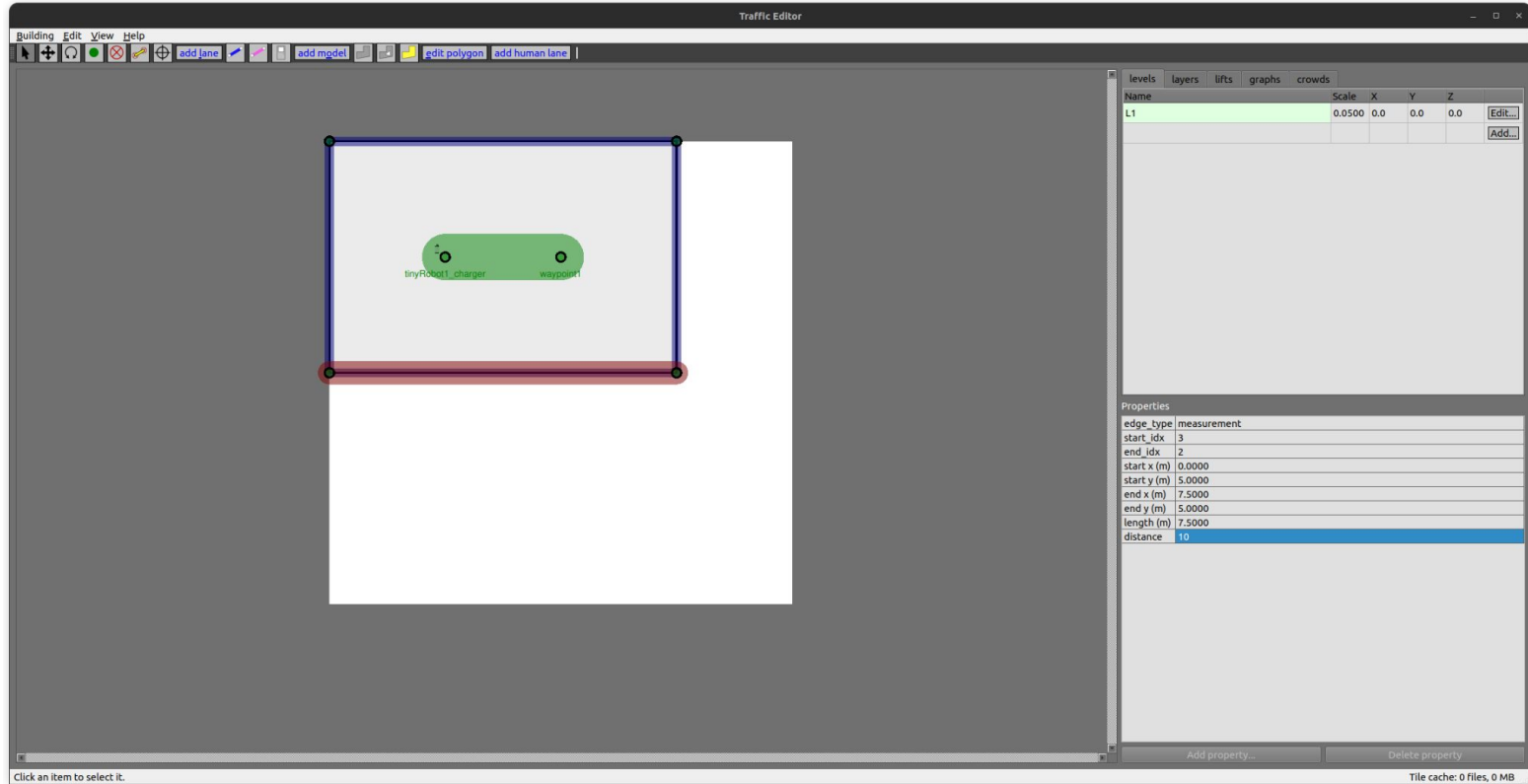
Add a lane



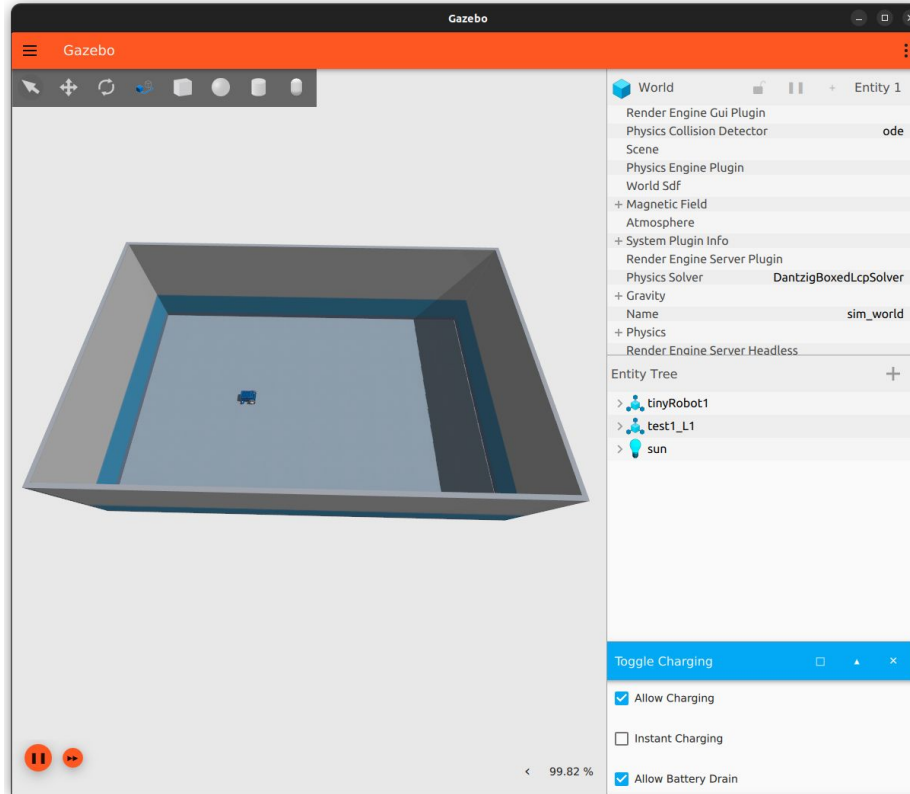
First launch



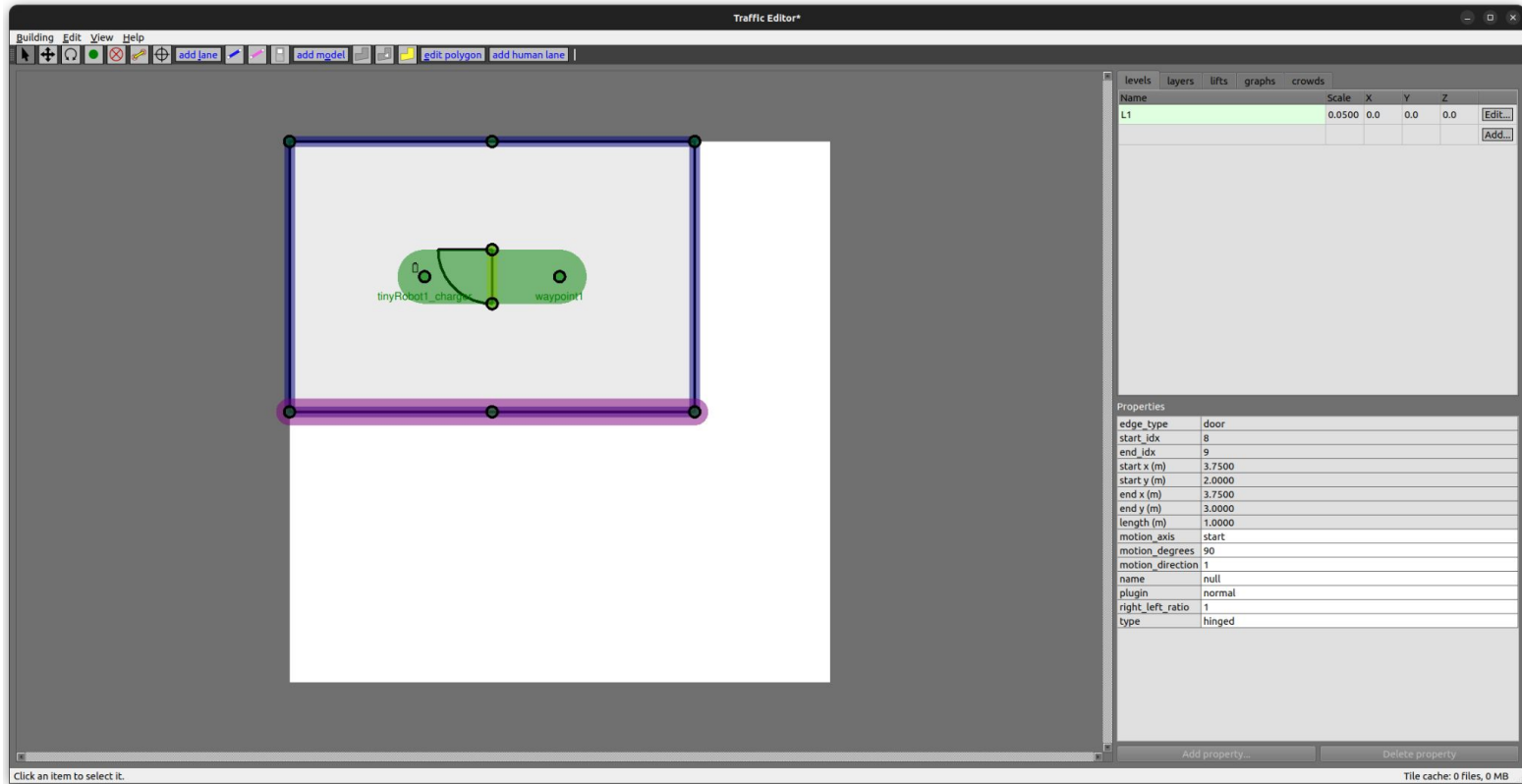
Set measurements



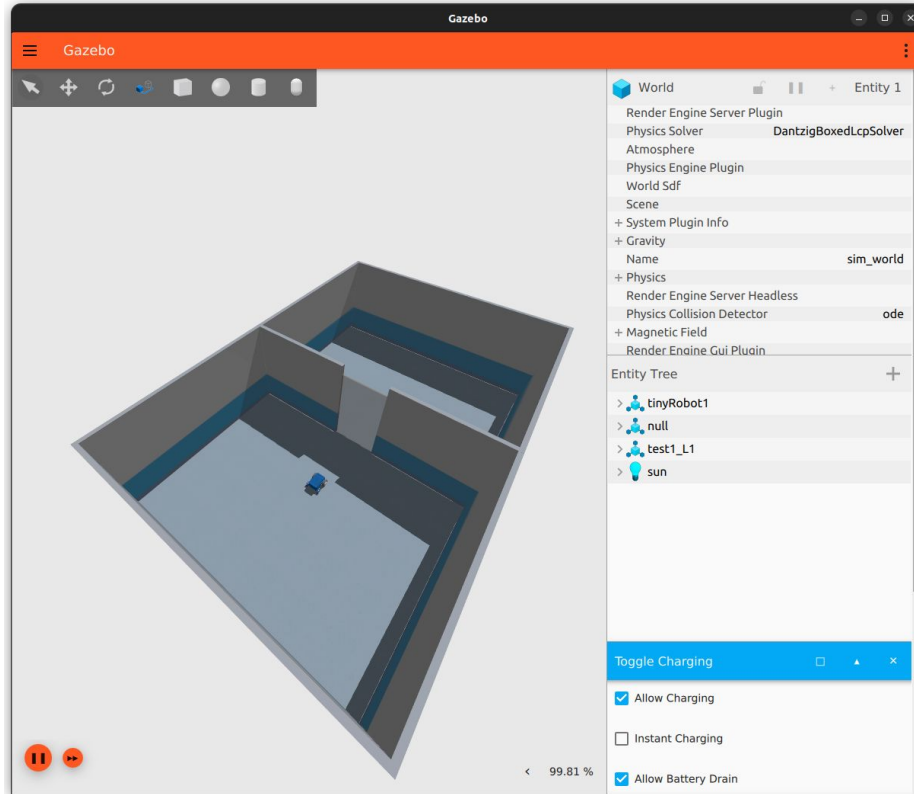
Measurement simulation



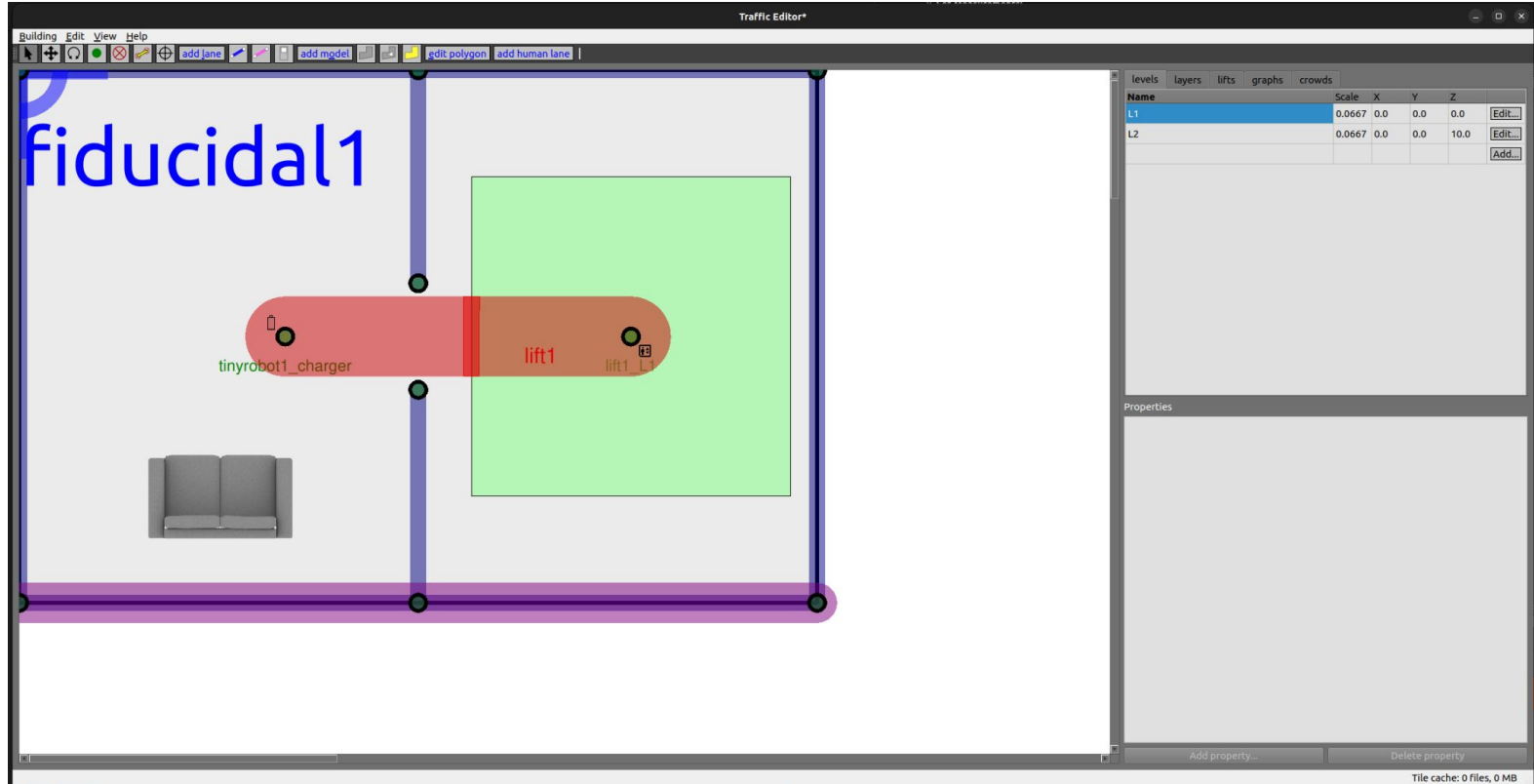
Add a door



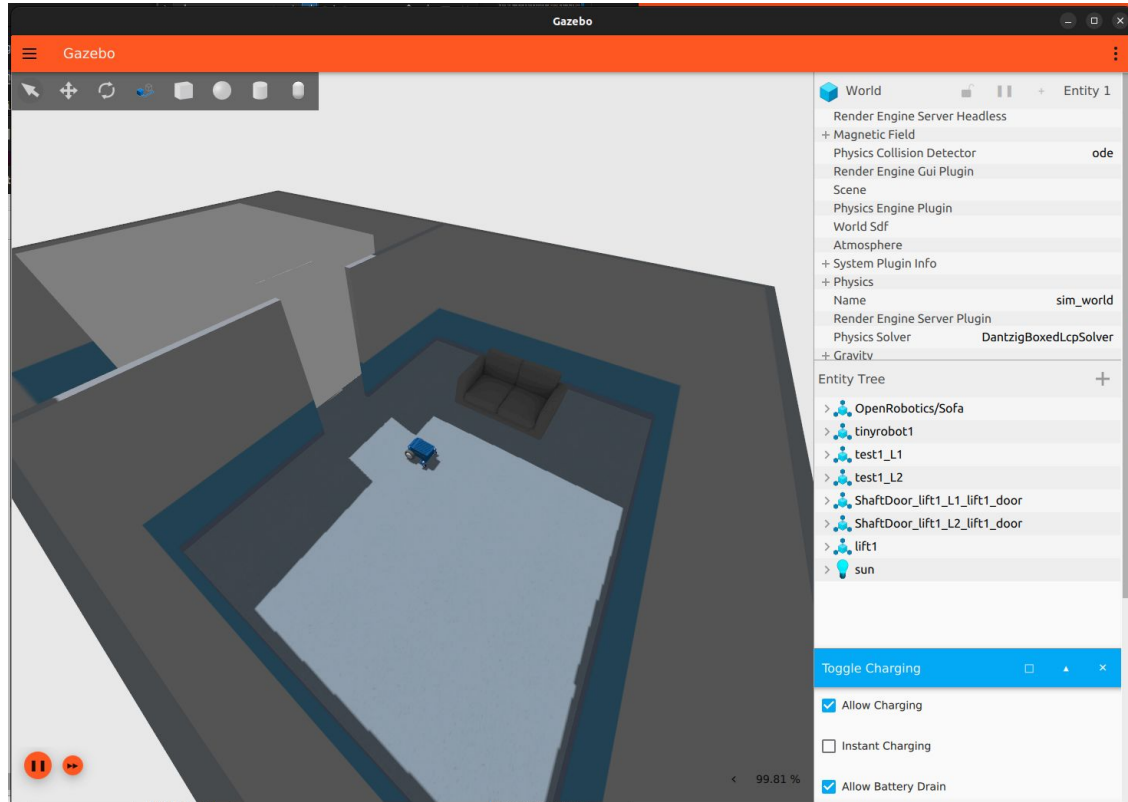
Door simulation



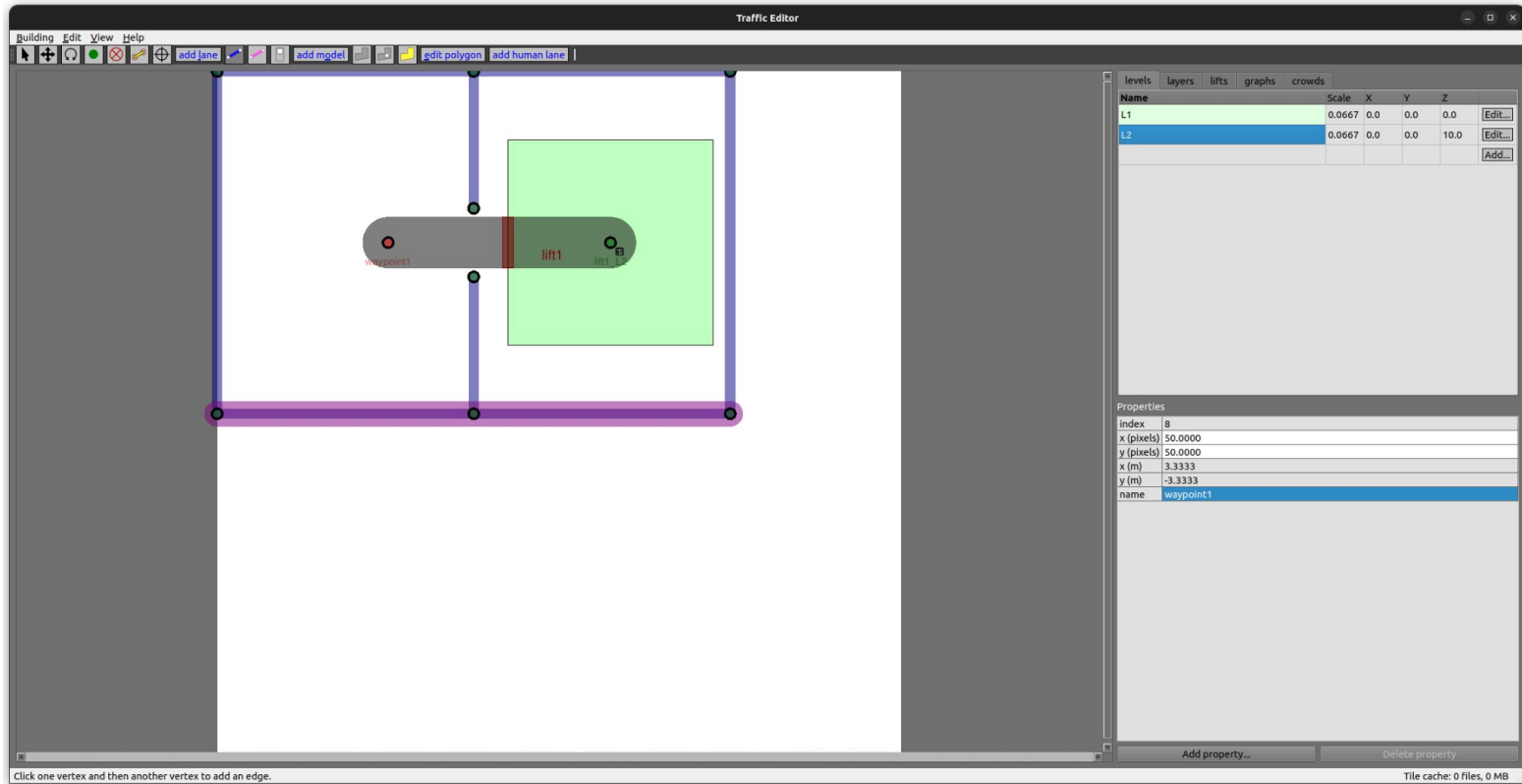
Add models



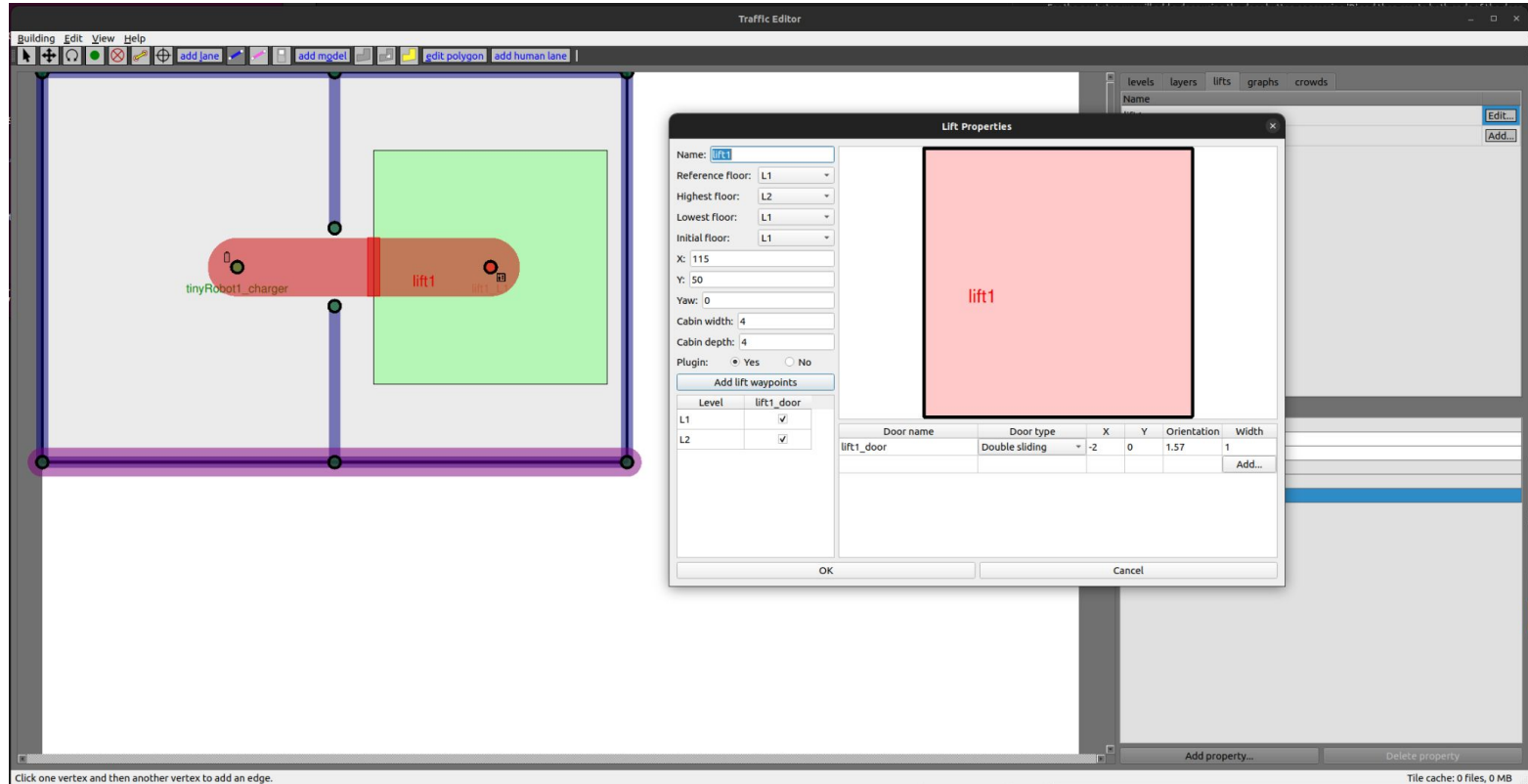
Simulation with models



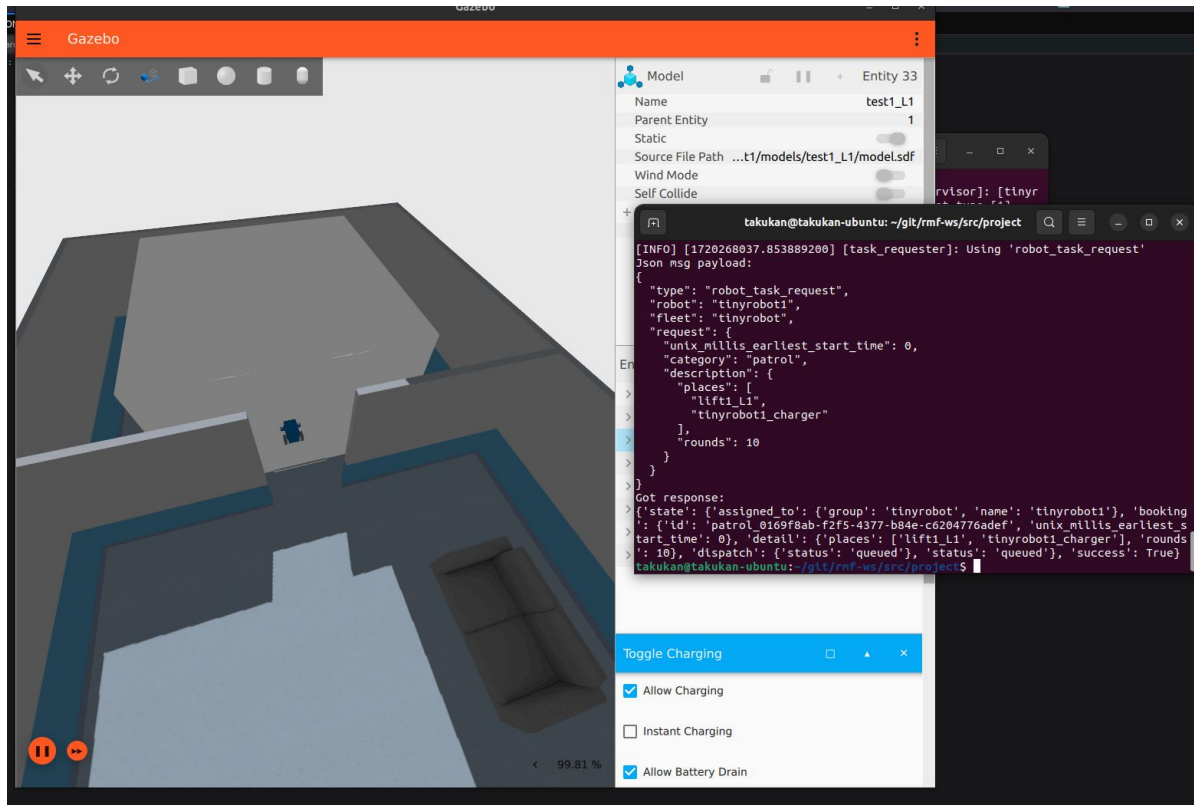
Add a second floor



Create a lift



Executing a task



Launch ICC_Kyoto world

```
rocker --nvidia --x11 \  
  -e ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST \  
  --network host --user \  
  --volume `pwd`/rmf_ws:/home/usuario/rmf_ws -- \  
  ghcr.io/open-rmf/rmf/rmf_demos:jazzy-rmf-latest \  
  bash
```

Launch ICC_Kyoto world

```
sudo cp -R /root/.gazebo .
```

```
cd rmf_ws/
```

```
source install/setup.bash
```

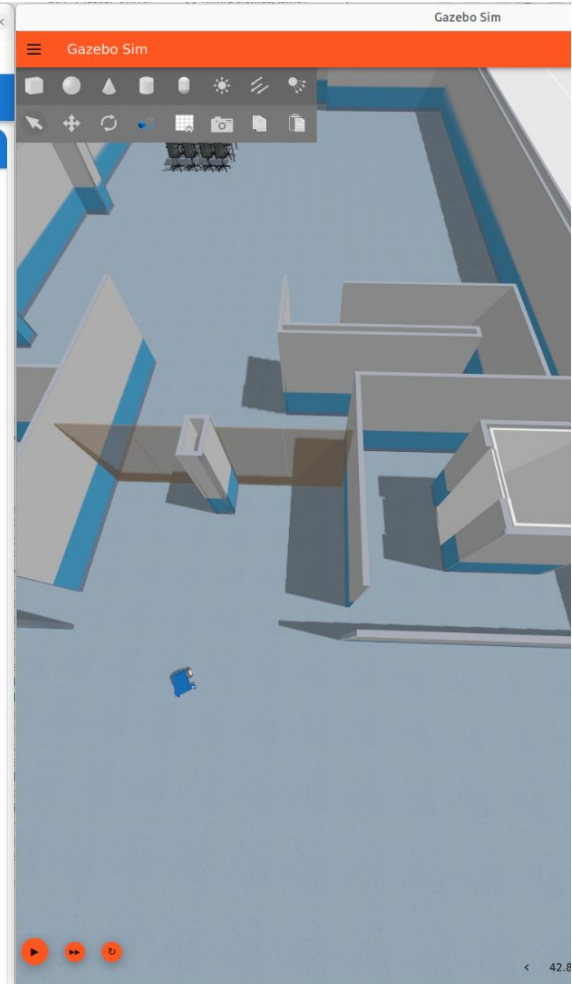
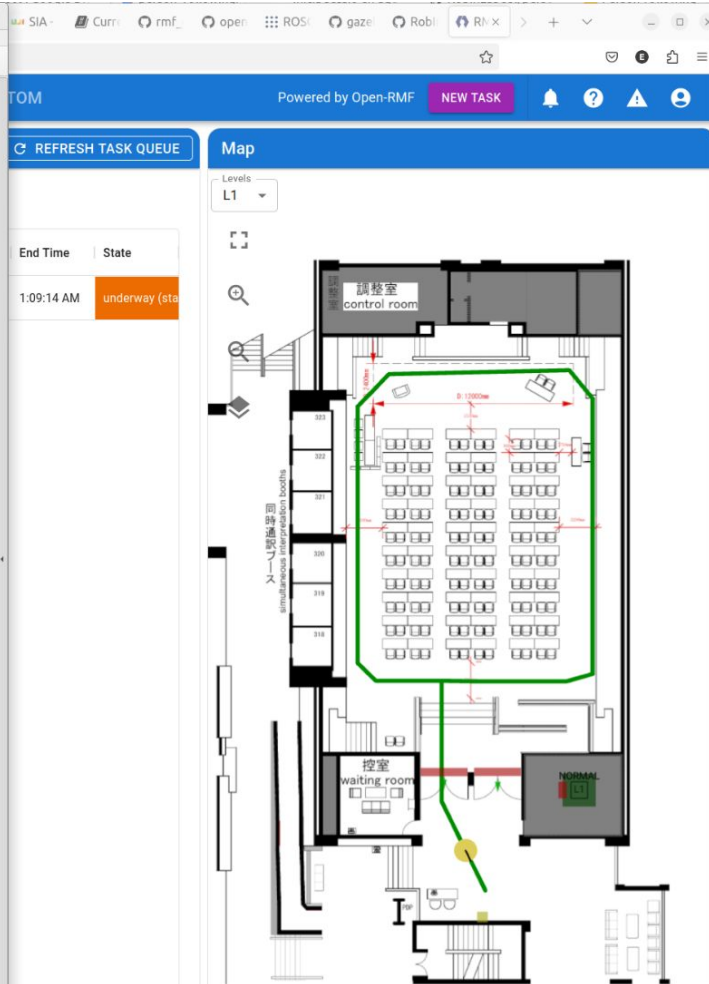
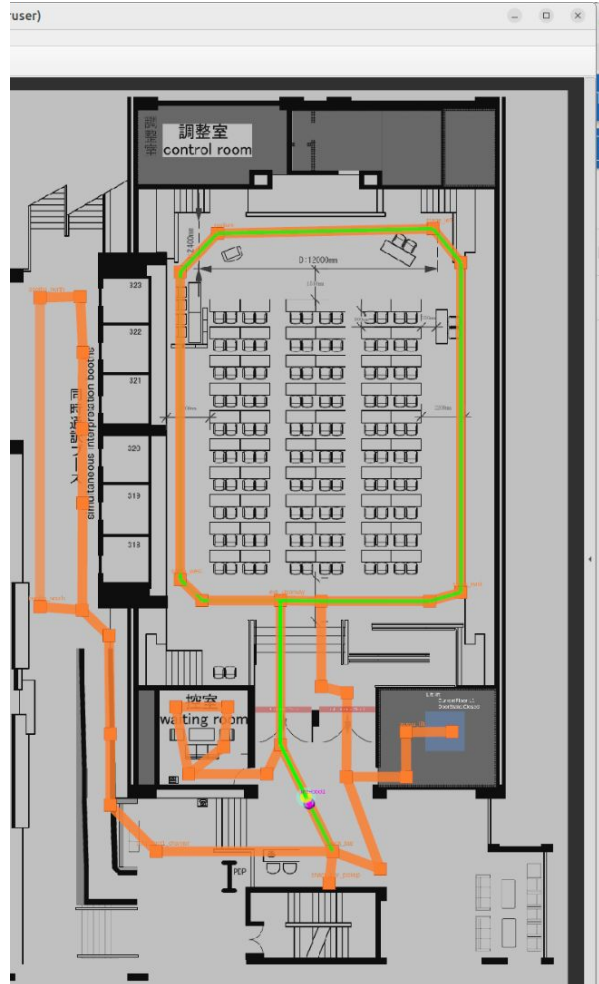
```
ros2 launch project_simulation \  
  icc_kyoto.launch.xml \  
  server_uri:="ws://localhost:8000/_internal"
```


Terminal 2 - API Server

```
docker run --network host -it \  
  -e ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST \  
  -e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \  
  ghcr.io/open-rmf/rmf-web/api-server:jazzy-nightly
```

Terminal 3 - dashboard

```
docker run --network host -it \  
  -e RMF_SERVER_URL=http://localhost:8000 \  
  -e TRAJECTORY_SERVER_URL=ws://localhost:8006 \  
  ghcr.io/open-rmf/rmf-web/dashboard:jazzy-nightly
```



Exercise in GitHub repository

IMPORTANT: include a README in your repository with all the necessary commands for launching the simulations of the simple world and the ICC Kyoto world, and add some screen captures with examples of tasks.

Use the [Open-RMF demos README](#) as example.

References

Romero Mollá, J. (2024). Practical guide to implement OpenRMF in multi-robot environments. Treball Final de Grau en Enginyeria Informàtica. Escola Politècnica Superior de la Universitat d'Alacant.

[RUA: Practical guide to implement OpenRMF in multi-robot environments](#)